

Informe de Laboratorio 05

Tema: Base de datos

Nota

Integrantes (Grupo 2)	Escuela	Asignatura
Salas Zegarra, Marco Antonio Molina Soncco, Percy Vicente Medina, Eder Lucio Peralta Llasa, Kevin	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: I Código: 1703122

Laboratorio	Tema	Duración
05	Base de datos	06 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	4 de mayo 2026	12 de mayo 2026

1. Objetivos del Laboratorio

- Diseñar el modelo lógico de una base de datos mediante un diagrama DER, identificando entidades, atributos y relaciones.
- Implementar el modelo físico de la base de datos utilizando PostgreSQL, aplicando sentencias SQL para la creación de tablas y restricciones.
- Integrar la base de datos en la plataforma *Supabase* como servicio backend en la nube, verificando su correcto funcionamiento.
- Documentar el desarrollo de la práctica mediante la elaboración de un archivo `README.md` y el uso de un repositorio en GitHub.

2. Entorno de Desarrollo y Herramientas

- **Sistema Operativo:** Windows 11 / Ubuntu.
- **Sistema de Gestión de Base de Datos:** PostgreSQL 16.x.
- **Plataforma Backend en la Nube:** Supabase.
- **Lenguaje de Consulta:** SQL.
- **Control de Versiones:** Git y GitHub.

- **Editor de Código:** Visual Studio Code.
- **Herramienta de Modelado:** Mermaid Database Visualizer para la elaboración del diagrama DER.

3. Desarrollo Técnico

3.1. Estructura del Proyecto Grupal

Se creó el repositorio grupal en GitHub para almacenar los archivos y avances del laboratorio. Posteriormente, se organizó la estructura del proyecto incluyendo el archivo del diagrama entidad-relación (DER) elaborado con Mermaid mediante el archivo `mermaid_code.txt`, el script SQL correspondiente a la implementación física en PostgreSQL y la documentación del proyecto en el archivo `README.md`.

Además, se realizó la integración de la base de datos en Supabase como servicio backend en la nube, importando el script SQL generado previamente y verificando el correcto funcionamiento de las tablas y relaciones implementadas.

Listing 1: Árbol de directorios del proyecto

```
proyecto-bd/  
|--- backend/  
| |--- .gitignore  
| |--- README.md  
| |--- requirements.txt  
|  
|--- bd/  
| |--- .gitignore  
| |--- README.md  
| |--- bd.sql  
|  
|--- frontend/  
| |--- .gitignore  
| |--- README.md  
| |--- package.json  
|  
|--- .gitignore  
|--- README.md
```

Figura 1: Estructura de directorios visualizada en el editor.

3.2. Diseño del Modelo Lógico DER

El diseño del modelo entidad-relación (DER) se enfocó en la construcción de un sistema de gestión de suscripciones digitales, permitiendo administrar usuarios, servicios, pagos, categorías y configuraciones de notificaciones. La entidad central del modelo es **SUBSCRIPTIONS**, la cual conecta a los usuarios con los servicios contratados y almacena la información principal de cada suscripción.

Asimismo, se definieron entidades complementarias como **USERS**, **SERVICES**, **CATEGORIES** y **CURRENCIES**, encargadas de organizar la información general del sistema. También se incorporaron tablas de historial como **PAYMENT_HISTORY** y **PRICE_HISTORY**, permitiendo mantener un registro de pagos y cambios de precio a lo largo del tiempo.

Finalmente, el modelo incluye funcionalidades adicionales para compartir suscripciones mediante **SUBSCRIPTION_SHARES**, gestionar deudas con **DEBTS** y configurar notificaciones utilizando las tablas **NOTIFICATION_CONFIGS** y **NOTIFICATION_LOGS**.

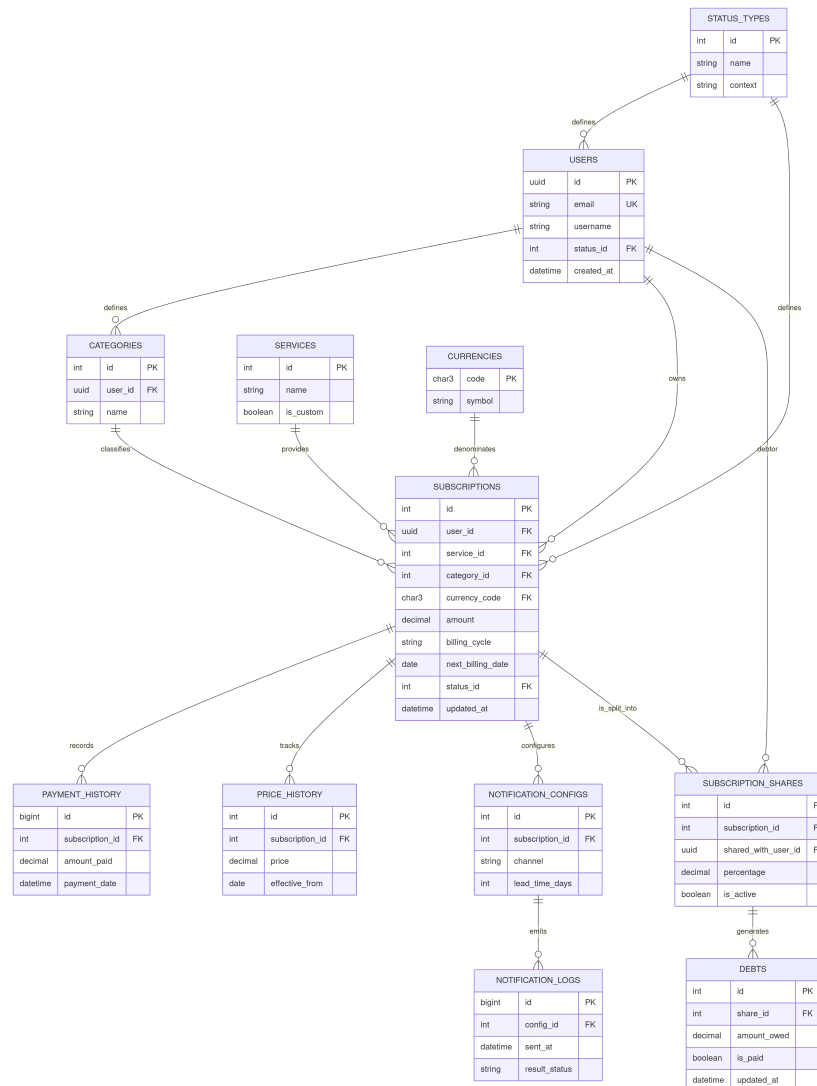


Figura 2: Diagrama entidad-relación (DER) del sistema

Listing 2: Descripción general de las principales tablas del modelo

USERS

Almacena la información principal de los usuarios registrados.

SERVICES

Contiene el catálogo de plataformas o servicios digitales.

SUBSCRIPTIONS

Tabla principal del sistema donde se registran las suscripciones.

CATEGORIES

Permite clasificar las suscripciones por tipo.

PAYMENT_HISTORY

Registra el historial de pagos realizados.

PRICE_HISTORY

Mantiene el historial de cambios de precio.

SUBSCRIPTION_SHARES

Administra el reparto de costos entre usuarios.

DEBTS

Controla las deudas pendientes relacionadas a suscripciones compartidas.

NOTIFICATION_CONFIGS

Configura recordatorios y alertas.

NOTIFICATION_LOGS

Registra el historial de notificaciones enviadas.

Criterios de Diseño Lógico y Normalización

En la fase de diseño lógico, se transformó el modelo conceptual en un esquema relacional normalizado, cumpliendo con la **Tercera Forma Normal (3NF)**. Esta normalización se evidencia en la segregación de los estados en la tabla `statusTypes` y la independencia de los catálogos de servicios y monedas, eliminando redundancias y anomalías de actualización.

Se aplicaron convenciones de nomenclatura técnica rigurosas:

- **Identidad y Auditoría:** Todas las tablas integran de forma nativa seis atributos de control: `id` (PK), `status`, `created`, `modified`, `created_id` y `modified_id`, asegurando que cada registro sea auditable desde su creación hasta su última modificación.
- **Nomenclatura:** Se utilizó el idioma inglés para la definición de objetos, empleando nombres en plural y el estilo *CamelCase* (protegido mediante comillas dobles en el DDL) para las tablas.
- **Integridad Referencial:** Las llaves foráneas siguen el estándar `tableName_id`, lo que facilita la legibilidad del código y la automatización de procesos de unión (*joins*).

3.3. Implementación física

La implementación física se realizó sobre el motor PostgreSQL, aprovechando tipos de datos avanzados para garantizar la precisión y la escalabilidad.

- **Gestión de Identidad:** Se optó por el uso de UUID (Universally Unique Identifier) para la tabla de usuarios, lo cual proporciona una capa de seguridad adicional al evitar la enumeración predecible de IDs en la capa de aplicación.
- **Precisión Financiera:** Los campos de costos y deudas utilizan el tipo `DECIMAL(10,2)`, evitando los errores de redondeo inherentes a los tipos de punto flotante en cálculos contables.
- **Resolución de Relaciones N:M:** La relación entre suscripciones y usuarios se resolvió mediante la tabla intermedia `subscriptions_users`, nombrada siguiendo el orden alfabético de las entida-

des relacionadas. Esta tabla es fundamental para el módulo de gastos compartidos, vinculándose directamente con la tabla de `debts` para el seguimiento de saldos pendientes.

- **Optimización de Carga:** Se utilizaron tipos `SERIAL` y `BIGSERIAL` para las llaves primarias de tablas de alto tráfico como historiales y logs, optimizando el rendimiento de indexación.

```

2  -- 1. TABLAS MAESTRAS (Con campos de auditoria exigidos)
3
4  Code view is read-only. Switch to the editor.
5  CREATE TABLE "statusTypes" (
6      id SERIAL PRIMARY KEY,
7      name VARCHAR(50) NOT NULL,
8      context VARCHAR(50) NOT NULL,
9      status VARCHAR(20) DEFAULT 'Active',
10     created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
11     modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
12     created_id UUID,
13     modified_id UUID
14 );
15
16 CREATE TABLE "currencies" (
17     id CHAR(3) PRIMARY KEY, -- Mantenemos el código como ID por el estándar ISO
18     symbol VARCHAR(5) NOT NULL,
19     status VARCHAR(20) DEFAULT 'Active',
20     created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
21     modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
22     created_id UUID,
23     modified_id UUID
24 );
25
26 CREATE TABLE "services" (
27     id SERIAL PRIMARY KEY,
28     name VARCHAR(100) NOT NULL,
29     isCustom BOOLEAN DEFAULT FALSE,
30     status VARCHAR(20) DEFAULT 'Active',
31     created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
32     modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
33     created_id UUID,
34     modified_id UUID
35 );
36
37 -- =====
38 -- 2. NÚCLEO DE USUARIOS
39 -- =====
40
41 CREATE TABLE "users" (
42     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
43     email VARCHAR(255) UNIQUE NOT NULL,
44     username VARCHAR(100),
45     status VARCHAR(20) DEFAULT 'Active',
46     created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
47     modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
48     created_id UUID,
49     modified_id UUID
50 );
51

```

Figura 3: Creación de tablas maestras y núcleo de usuarios en PostgreSQL.

```

52  -- =====
53  -- 3. SUSCRIPCIONES Y CLASIFICACIÓN
54  -- =====
55
56  CREATE TABLE "categories" (
57      id SERIAL PRIMARY KEY,
58      user_id UUID REFERENCES "users"(id) ON DELETE CASCADE,
59      name VARCHAR(100) NOT NULL,
60      status VARCHAR(20) DEFAULT 'Active',
61      created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
62      modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
63      created_id UUID,
64      modified_id UUID
65  );
66
67  CREATE TABLE "subscriptions" (
68      id SERIAL PRIMARY KEY,
69      user_id UUID REFERENCES "users"(id) ON DELETE CASCADE,
70      service_id INT REFERENCES "services"(id),
71      category_id INT REFERENCES "categories"(id) ON DELETE SET NULL,
72      currency_id CHAR(3) REFERENCES "currencies"(id),
73      amount DECIMAL(10, 2) NOT NULL,
74      billingCycle VARCHAR(50) NOT NULL,
75      nextBillingDate DATE NOT NULL,
76      status VARCHAR(20) DEFAULT 'Active',
77      created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
78      modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
79      created_id UUID,
80      modified_id UUID
81  );
82
83  -- =====
84  -- 4. HISTORIAL Y AUDITORÍA
85  -- =====
86
87  CREATE TABLE "paymentHistories" (
88      id BIGSERIAL PRIMARY KEY,
89      subscription_id INT REFERENCES "subscriptions"(id) ON DELETE CASCADE,
90      amountPaid DECIMAL(10, 2) NOT NULL,
91      paymentDate TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
92      status VARCHAR(20) DEFAULT 'Active',
93      created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
94      modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
95      created_id UUID,
96      modified_id UUID
97  );
98

```

Figura 4: Implementación de suscripciones y relaciones principales del sistema.

```
99  CREATE TABLE "priceHistories" (  
100      id SERIAL PRIMARY KEY,  
101      subscription_id INT REFERENCES "subscriptions"(id) ON DELETE CASCADE,  
102      price DECIMAL(10, 2) NOT NULL,  
103      effectiveFrom DATE NOT NULL,  
104      status VARCHAR(20) DEFAULT 'Active',  
105      created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
106      modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
107      created_id UUID,  
108      modified_id UUID  
109  );  
110  
111  -- =====  
112  -- 5. COMPARTICIÓN (M:N Ordenada Alfabéticamente) Y DEUDAS  
113  -- =====  
114  
115  -- Antes era subscriptionShares, ahora cumple la regla N:M (subscriptions + users)  
116  CREATE TABLE "subscriptions_users" (  
117      id SERIAL PRIMARY KEY,  
118      subscription_id INT REFERENCES "subscriptions"(id) ON DELETE CASCADE,  
119      user_id UUID REFERENCES "users"(id) ON DELETE CASCADE,  
120      percentage DECIMAL(5, 2) NOT NULL CHECK (percentage > 0 AND percentage <= 100),  
121      status VARCHAR(20) DEFAULT 'Active',  
122      created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
123      modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
124      created_id UUID,  
125      modified_id UUID  
126  );  
127  
128  CREATE TABLE "debts" (  
129      id SERIAL PRIMARY KEY,  
130      subscriptions_user_id INT REFERENCES "subscriptions_users"(id) ON DELETE CASCADE,  
131      amountOwed DECIMAL(10, 2) NOT NULL,  
132      isPaid BOOLEAN DEFAULT FALSE,  
133      status VARCHAR(20) DEFAULT 'Active',  
134      created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
135      modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
136      created_id UUID,  
137      modified_id UUID  
138  );  
139
```

Figura 5: Configuración de historiales, compartición de suscripciones y deudas.

```
140  -- =====
141  -- 6. NOTIFICACIONES
142  -- =====
143
144  CREATE TABLE "notificationConfigs" (
145      id SERIAL PRIMARY KEY,
146      subscription_id INT REFERENCES "subscriptions"(id) ON DELETE CASCADE,
147      channel VARCHAR(50) NOT NULL,
148      leadTimeDays INT NOT NULL,
149      status VARCHAR(20) DEFAULT 'Active',
150      created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
151      modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
152      created_id UUID,
153      modified_id UUID
154  );
155
156  CREATE TABLE "notificationLogs" (
157      id BIGSERIAL PRIMARY KEY,
158      notificationConfig_id INT REFERENCES "notificationConfigs"(id) ON DELETE CASCADE,
159      sentAt TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
160      resultStatus VARCHAR(50) NOT NULL,
161      status VARCHAR(20) DEFAULT 'Active',
162      created TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
163      modified TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
164      created_id UUID,
165      modified_id UUID
166  );
167
168  -- =====
169  -- 7. SEGURIDAD (RLS SUPABASE)
170  -- =====
171
172  ALTER TABLE "subscriptions" ENABLE ROW LEVEL SECURITY;
173
174  CREATE POLICY "Usuarios ven sus propias suscripciones"
175  ON "subscriptions" FOR ALL
176  USING (auth.uid() = user_id);
```

Figura 6: Implementación de notificaciones y políticas de seguridad en PostgreSQL.

3.4. Implementación en la nube

La fase final del proyecto consistió en trasladar la base de datos de un entorno de desarrollo local a una infraestructura gestionada en la nube utilizando **Supabase**. Esta plataforma permite que el esquema de PostgreSQL sea accesible de forma remota, eliminando la necesidad de gestionar servidores físicos o configuraciones complejas de red.

El proyecto implementado en Supabase puede visualizarse mediante el siguiente enlace:

<https://supabase.com/dashboard/project/zdjppofxtfnfbjagjhbja>

Seguridad y Aislamiento de Datos

El aspecto más relevante de esta implementación fue la configuración de la **Seguridad a Nivel de Fila (RLS)**. En una base de datos convencional, una vez que se obtiene acceso, generalmente

se pueden ver todos los registros de una tabla. Sin embargo, mediante RLS, la seguridad se integra directamente en el motor de la base de datos:

- **Políticas de Acceso Granular:** Se implementó una política que utiliza la función nativa `auth.uid()` para identificar al usuario que realiza la consulta.
- **Filtro Automático:** El sistema actúa como un filtro invisible: cuando un usuario solicita sus suscripciones, el motor de la base de datos verifica su identidad y le entrega únicamente los registros vinculados a su identificador único.
- **Privacidad por Diseño:** Este enfoque garantiza que la información financiera y personal permanezca aislada, asegurando que ningún usuario pueda acceder a los datos de otros, incluso si intentan realizar consultas directas a la base de datos.

Disponibilidad y Escalabilidad

Al utilizar un modelo de **Backend as a Service (BaaS)**, el sistema aprovecha la infraestructura de la nube para garantizar que los datos estén disponibles en todo momento. Esto simplifica futuras expansiones del proyecto, como la conexión de una aplicación móvil o web, manteniendo una base de datos centralizada y segura que cumple con los estándares técnicos actuales.

3.5. Control de Versiones y Trabajo Colaborativo (Commits Clave)

El desarrollo del proyecto se gestionó de manera colaborativa utilizando Git y GitHub. El historial refleja una progresión ordenada por parte de un miembro del equipo; y finalmente algunas correcciones y actualizaciones mínimas.

A continuación, se detalla la secuencia de comandos de control de versiones ejecutados para alcanzar la versión final del repositorio:

Listing 3: Secuencia de comandos Git para el trabajo colaborativo

```
$ git add .
$ git commit -m "Create README.md"

$ git add .
$ git commit -m "Create backend"

$ git add .
$ git commit -m "Delete backend"

$ git add .
$ git commit -m "estructura base"

$ git add .
$ git commit -m "Merge pull request #1 from evicente3/evicentem"

$ git add .
$ git commit -m "Llenando README"

$ git add .
$ git commit -m "Merge branch 'evicentem'"

$ git add .
$ git commit -m "Base de readme general"
```

```
$ git add .
$ git commit -m "Merge branch 'evicentem'"
```

3.6. Resumen de Accesos y Evidencias Digitales

Con el objetivo de facilitar la revisión y validación de los entregables desarrollados durante la práctica de laboratorio, se presentan las principales evidencias digitales relacionadas con el proyecto. Estas evidencias incluyen el repositorio grupal en GitHub, donde se almacenaron los archivos del sistema y la documentación del proyecto, así como la implementación de la base de datos en Supabase mediante la importación del script SQL desarrollado en PostgreSQL.

Las capturas presentadas permiten verificar la estructura general del repositorio, la organización de los módulos del proyecto y la correcta creación de las tablas y relaciones dentro del entorno cloud proporcionado por Supabase.

1. **Repositorio en GitHub (Código Fuente y Versionado):** Espacio de trabajo colaborativo donde se gestionó el historial de commits y la integración de las ramas.

<https://github.com/evicente3/MGS/tree/main/bd>

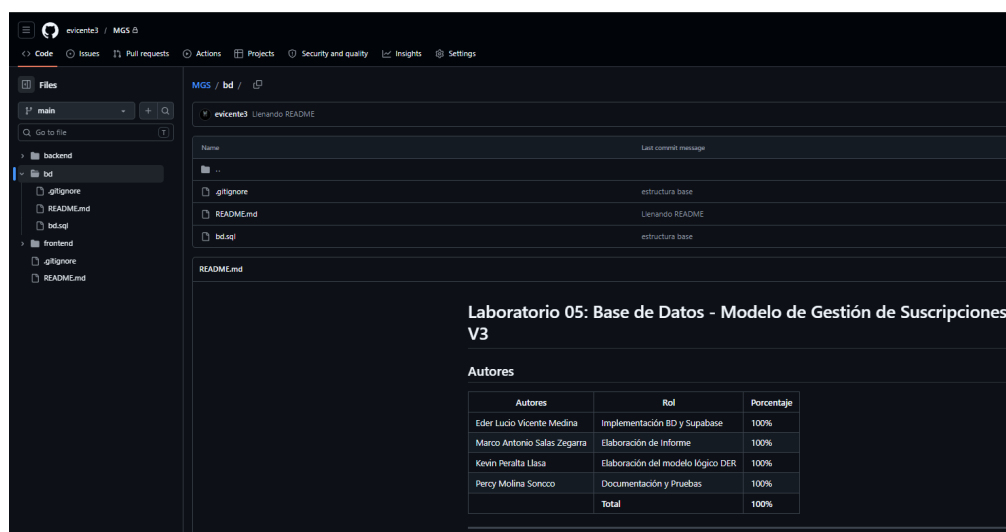


Figura 7: Captura de pantalla del repositorio oficial en GitHub.

2. **Implementación de la Base de Datos en Supabase:** Visualización de las tablas y relaciones implementadas en Supabase mediante la herramienta *Schema Visualizer*, verificando la correcta importación del modelo físico desarrollado en PostgreSQL.

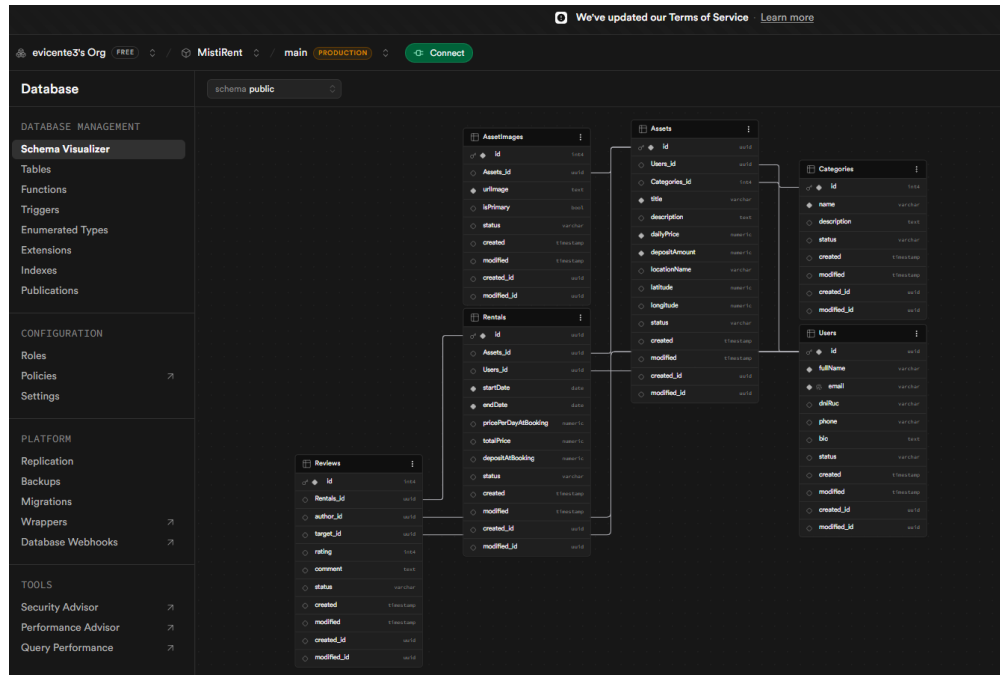


Figura 8: Captura de pantalla de la base de datos implementada en Supabase.

4. Conclusiones

- El desarrollo del modelo entidad-relación permitió comprender la importancia del diseño lógico en la organización y estructuración de una base de datos, facilitando la correcta definición de entidades, atributos y relaciones antes de la implementación física.
- La implementación del modelo físico en PostgreSQL permitió aplicar conocimientos de SQL para la creación de tablas, claves primarias y claves foráneas, garantizando la integridad y consistencia de la información almacenada.
- La integración de la base de datos en Supabase demostró la utilidad de las plataformas backend en la nube para administrar bases de datos de manera remota, simplificando el acceso, despliegue y gestión del sistema desarrollado.

5. Rúbricas

5.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

5.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumple con el ítem correspondiente.
- El alumno debe autocalificarse en la columna **Estudiante**.

Tabla 2: Rúbrica para contenido del informe y demostración

Ítem	Descripción	Puntaje
DER	Elaboración del modelo lógico DER.	4
Modelo físico	Implementación del modelo físico PostgreSQL. Incluye URL en YouTube, informe y README.md.	7
Supabase	Implementación en Supabase.	4
Informe	El laboratorio tiene un README.md que detalla toda la práctica.	3
Prueba	Se tomaron en cuenta todas las consideraciones y recomendaciones, lo que evidencia un trabajo en equipo.	-0
Total		18